📦 Dynamic Order & Logistics Management System

A production-grade, highly flexible Order Management & Logistics Engine built with Python. This project demonstrates modern **Object-Oriented Programming (OOP)** practices, focusing on **Composition over Inheritance**, the **Strategy Design Pattern**, and **Dynamic Runtime Service Mutation**.

🎯 Executive Summary

In enterprise e-commerce systems, using class inheritance to model combinations of payment methods, carriers, and discount rules leads to **Class Explosion** (e.g., `AramexCreditCardDiscountedOrder`).

This system solves the architecture bottleneck by designing a **Dynamic Container Architecture**:

- Orders maintain dynamic registries for attributes and runtime behaviors.
- Service strategies (Shipping Providers, Payment Gateways, Discount Engines) are injected at runtime via standardized interface contracts.
- The system supports **Runtime Service Mutation**, allowing an order to switch shipping carriers or payment strategies dynamically without modifying core business logic.

🏗️ System Architecture & Design Patterns

1. Composition over Inheritance (`Has-A` Relationship)

Orders do not inherit from logistics or payment providers. Instead, an order holds references to interchangeable service objects in its internal attribute dictionary.

2. Strategy Pattern

Algorithms for shipping cost calculation, payment processing, and promotional rules are encapsulated behind clear interface contracts:

- `IShippingProvider`: Encapsulates carrier-specific logic (`AramexShipping`, `BostaShipping`).

- `IPaymentStrategy`: Encapsulates payment gateway behaviors (`VodafoneCashPayment`, `CreditCardPayment`).
- `IDiscountRule`: Encapsulates pricing rules (`PromoCodeDiscount`, `NoDiscount`).

3. Abstract Base Classes (ABCs) & Polymorphism

All service strategies inherit from explicit Abstract Base Classes, guaranteeing method signatures (`calculate_cost`, `ship`, `pay`, `apply_discount`) and enforcing interface contracts across all concrete implementations.

4. Data Encapsulation & Security

Sensitive parameters—such as financial account numbers and masked credit card credentials—are encapsulated inside private attributes (`__phone_number`, `__card_number`).

💻 Tech Stack & Environment

- **Language:** Python 3.x
- **Core Modules:** `abc` (Abstract Base Classes)
- **Environment:** Compatible with Visual Studio Code, Jupyter Notebook, and Standalone Python Runtimes.

⚙️ Core Logic Flow

[DynamicOrder]

      |

      ├──► attributes["shipping_provider"] ---> IShippingProvider (Aramex / Bosta)

      ├──► attributes["payment_strategy"]   ---> IPaymentStrategy (Vodafone Cash / Credit Card)

      └──► attributes["discount_rule"]   ---> IDiscountRule   (PromoCode / NoDiscount)

      |

      ▼

process_order()

    |

    ├── 1. Compute Subtotal & Weight

    ├── 2. Apply Discount Strategy

    ├── 3. Calculate Shipping Strategy Cost

    ├── 4. Execute Payment Strategy Transaction

    └── 5. Trigger Shipping Strategy Fulfillment

🚀 Execution Scenarios & Sample Output

The system demonstrates **Runtime Service Mutation** by swapping services on the fly during order processing:

=== [Scenario 1: Initial Setup] ===

--- Processing Dynamic Order #ORD-2026-X ---

--> Applied Promo Code 'SAVE15' (15.0% OFF: -300.00 EGP)

Subtotal: $2000.0 | After Discount: $1700.0 | Shipping: $65.0 | Total: $1765.0

--> Initiating Vodafone Cash transaction of 1765.00 EGP for 01012345678...

--> OTP Verified. Payment Successful!

Success: [Aramex Express] Shipment waybill created for order 'ORD-2026-X'.

=== [Scenario 2: Runtime Service Mutation] ===

--- Processing Dynamic Order #ORD-2026-X ---

Subtotal: $2000.0 | After Discount: $2000.0 | Shipping: $42.0 | Total: $2042.0

--> Charging 2042.00 EGP to Credit Card (**** **** **** 4444) for Mohamed Hassan...

--> Bank Response: APPROVED.

Success: [Bosta Deliveries] Order 'ORD-2026-X' dispatched to local hub.

🛠️ How to Run

1. Clone or download this repository.
2. Open `OOP_PJ.1_2.ipynb` in VS Code or Jupyter Notebook.
3. Run all cells sequentially to observe the dynamic execution output.

👤 Author

Mohamed Hassan
MIS Student | Data Analytics & Business Intelligence Enthusiast
🔗 LinkedIn Profile